

Ex. No.: 9
Date: 02/04/25

DEADLOCK AVOIDANCE

Aim:

To find out a safe sequence using Banker's algorithm for deadlock avoidance.

Algorithm:

1. Initialize work=available and finish[i]=false for all values of i
2. Find an i such that
both: finish[i]=false
and Need_i ≤ work
3. If no such i exists go to step 6
4. Compute work=work+allocation_i
5. Assign finish[i] to true and go to step 2
6. If finish[i]=true for all i, then print safe sequence
7. Else print there is no safe sequence

Program Code:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int P, R;
    printf("Enter number of processes: ");
    scanf("%d", &P);
    printf("Enter number of resource types: ");
    scanf("%d", &R);

    int alloc[P][R], max[P][R], avail[R], need[P][R];
    bool finished[P];
    int safeSeq[P];

    // Input Allocation Matrix
    printf("Enter Allocation Matrix (%d x %d):\n", P, R);
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            scanf("%d", &alloc[i][j]);

    // Input Max Matrix
    printf("Enter Max Matrix (%d x %d):\n", P, R);
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            scanf("%d", &max[i][j]);

    // Input Available Resources
    printf("Enter Available Resources (%d):\n", R);
    for (int i = 0; i < R; i++)
        scanf("%d", &avail[i]);

    // Calculate Need Matrix and initialize finished[]
```

```

for (int i = 0; i < P; i++) {
    finished[i] = false;
    for (int j = 0; j < R; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int work[R];
for (int i = 0; i < R; i++)
    work[i] = avail[i];

int count = 0;
while (count < P) {
    bool found = false;
    for (int i = 0; i < P; i++) {
        if (!finished[i]) {
            bool possible = true;
            for (int j = 0; j < R; j++) {
                if (need[i][j] > work[j]) {
                    possible = false;
                    break;
                }
            }
            if (possible) {
                for (int j = 0; j < R; j++)
                    work[j] += alloc[i][j];
                safeSeq[count++] = i;
                finished[i] = true;
                found = true;
            }
        }
    }
    if (!found) {
        printf("\nSystem is not in a safe state.\n");
        return 1;
    }
}

// Print Safe Sequence
printf("\nSystem is in a safe state.\nSafe Sequence: ");
for (int i = 0; i < P; i++)
    printf("P%d ", safeSeq[i]);
printf("\n");

return 0;
}

```

Sample Output:

The SAFE Sequence is

P1 -> P3 -> P4 -> P0 -> P2

Result:

Thus the program is compiled and executed successfully.